Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-210БВ-25

Студент: Баранова Д.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 28.10.26

Москва, 2026

Постановка задачи

Вариант 2.

Составить программу, которая сортирует массив целых чисел при помощи параллельного алгоритма быстрой сортировки на языке Си. Программа должна обрабатывать данные в многопоточном режиме. При обработки использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы. Так же необходимо уметь продемонстрировать количество потоков, используемое вашей программой с помощью стандартных средств операционной системы.

Общий метод и алгоритм решения

В лабораторной работе реализована сортировка массива целых чисел с использованием параллельного алгоритма быстрой сортировки на языке C. Для создания и управления потоками использована библиотека POSIX Threads (pthread).

Использованные системные средства и функции:

- pthread_create() – создание нового потока для выполнения части сортировки (сортировки правой половины разбиения).
- pthread_join() – ожидание завершения созданного потока; выступает барьером, гарантируя, что воркер завершён до возврата токена.
- sem_init() – инициализация семафора с начальным значением threads-1 (число разрешений на создание дополнительных потоков).
- sem_trywait() – неблокирующая попытка взять токен; при отсутствии токена текущий поток сортирует обе части самостоятельно, что исключает deadlock.
- sem_post() – возврат токена после завершения потока (после pthread_join), что обеспечивает соблюдение лимита одновременно работающих потоков.
- sem_destroy() – уничтожение семафора после завершения параллельной сортировки.

Алгоритм работы программы:

1. Программа считывает массив из файла input.txt и проверяет аргумент командной строки, задающий максимальное количество потоков.
2. Выполняет последовательную быструю сортировку и измеряет время её работы.
3. Выполняет параллельную быструю сортировку: массив разделяется на части, правая сортируется в новом потоке, левая в текущем.
4. Для ограничения количества одновременно работающих потоков используется семафор. Если лимит достигнут, то поток сортирует обе части.
5. Программа ожидает завершения потоков, измеряет время работы последовательного и параллельного алгоритма, записывает результат в result.txt.
6. Вычисляет ускорение и эффективность параллельной сортировки, выводит результаты в консоль.

Код программы

sort.cpp

```
#include <algorithm>
#include <chrono>
#include <cstdio>
#include <cstdlib>
#include <string>
#include <vector>
#include <pthread.h>
#include <semaphore.h>
#include <fstream>

static bool read_array_from_file(const std::string& path, std::vector<int>& a) {
        std::ifstream f(path);
        if (!f) {
            std::fprintf(stderr, "Cannot open file: %s\n", path.c_str());
            return false;
        }
        a.clear();
        int x;
        while (f >> x) {
            a.push_back(x);
        }
        if (!f.eof()) {
              std::fprintf(stderr, "Malformed number in file: %s\n", path.c_str());
              return false;
        }
        if (a.empty()) {
              std::fprintf(stderr, "File is empty: %s\n", path.c_str());
              return false;
        }
```

```
        return true;
}
static bool write_array_to_file(const std::string& path, const std::vector<int>& a) {
        std::ofstream f(path);
        if (!f) {
            std::fprintf(stderr, "Cannot open for writing: %s\n", path.c_str());
            return false;
        }
        for (int x : a) f << x << '\n';
        return f.good();
}
static double now_sec(){
    using namespace std::chrono;
    return duration<double>(steady_clock::now().time_since_epoch()).count();
}

static void insertion_sort(int* a, int l, int r){
    for (int i = l+1; i < r; i++){
        int x = a[i];
        int j = i-1;
        while ((j >= l) && (a[j] > x)){
            a[j+1] = a[j];
            j--;
        }
        a[j+1] = x;
    }
}

static void partition3(int* a, int l, int r, int& lt, int& gt, int pivot){
    int i = l;
     int less = l, greater = r;
     while (i < greater){
         if (a[i] < pivot){
             std::swap(a[less], a[i]);
             less++; i++;
         } else if (a[i] > pivot){
```

```
            greater--;
            std::swap(a[greater], a[i]);
        } else {
            i++;
        }
    }
    lt = less;
    gt = greater;
}

static int choose_pivot(const int *a, int l, int r) {
    int m = l + (r - l) / 2;
    int x = a[l], y = a[m], z = a[r - 1];
    if ((x <= y && y <= z) || (z <= y && y <= x)) return y;
    if ((y <= x && x <= z) || (z <= x && x <= y)) return x;
    return z;
}

static void seq_quicksort(int* a, int l, int r){
    while (r - l > 1){
        if (r - l < 32){
            insertion_sort(a, l, r);
            return;
        }
        int lt, gt;
        int pivot = choose_pivot(a, l, r);
        partition3(a, l, r, lt, gt, pivot);
        if (lt - l < r - gt){
            seq_quicksort(a, l, lt);
            l = gt;
        } else{
            seq_quicksort(a, gt, r);
            r = lt;
        }
    }
}
```

```
static bool is_sorted(const std::vector<int>& a){
    for (size_t i = 1; i < a.size(); i++){
        if (a[i-1] > a[i]) return false;
    }
    return true;
}

static const int kSeqThreshold = 4096;
static sem_t sem;

struct ThreadArg {
    int *a;
    int l;
    int r;
};

static void par_sort(int*a, int l, int r);

static void* worker(void* p){
    ThreadArg arg = *(ThreadArg*) p;
    free(p);
    par_sort(arg.a, arg.l, arg.r);
    return nullptr;
}

static void par_sort(int*a, int l, int r){
    if (r - l <= 1) return;
    if (r - l < kSeqThreshold){
        seq_quicksort(a, l, r);
        return;
    }
    int lt, gt;
    int pivot = choose_pivot(a, l, r);
    partition3(a, l, r, lt, gt, pivot);
```

```
        pthread_t tid;
        bool spawned = false;
        if (sem_trywait(&sem) == 0){
            ThreadArg* arg = (ThreadArg*)malloc(sizeof(ThreadArg));
            if (arg){
                arg->a = a;
                arg->l = gt;
                arg->r = r;
                if (pthread_create(&tid, nullptr, worker, arg) == 0){
                    spawned = true;
                } else{
                    free(arg);
                    sem_post(&sem);
                }
            } else {
                sem_post(&sem);
            }
        }

        par_sort(a, l, lt);

        if(spawned){
            pthread_join(tid, nullptr);
            sem_post(&sem);
        } else{
            par_sort(a, gt, r);
        }
}

static void par_quicksort(int* a, int n, int threads){
        if (n <= 1) return;
        if (threads <= 1) {
            seq_quicksort(a, 0, n);
            return;
        }
        sem_init(&sem, 0, threads - 1);
```

```
    par_sort(a, 0, n);
    sem_destroy(&sem);
}



int main(int argc, char** argv) {
    std::string input_file;
    int threads = 1;
    for (int i = 1; i < argc; ++i) {
        std::string k = argv[i];
        if (k == "-f"){
            if (i + 1 >= argc) { std::fprintf(stderr, "Missing value for -f\n"); return EXIT_FAILURE;}
            input_file = argv[++i];
        } else if (k == "-t"){
            if (i + 1 >= argc) { std::fprintf(stderr, "Missing value for -t\n"); return EXIT_FAILURE; }
            threads = std::atoi(argv[++i]);
        } else {
            std::fprintf(stderr, "Unknown argument: %s\n", k.c_str());
            return EXIT_FAILURE;
        }
    }
    if (input_file.empty() || threads <= 0) return 1;
    std::vector<int>original;
    if (!read_array_from_file(input_file, original)) return EXIT_FAILURE;
    int n =(int)original.size();

    std::vector<int> work(original);

    double t0 = now_sec();
    seq_quicksort(work.data(), 0, n);
    double t1 = now_sec();
    double T_seq = t1 - t0;

    if (!is_sorted(work)){
        std::fprintf(stderr, "seq: not sorted\n");
        return EXIT_FAILURE;
```

```
    }

    work = original;
    t0 = now_sec();
    par_quicksort(work.data(), n, threads);
    t1 = now_sec();
    double T_par = t1 - t0;

    if (!is_sorted(work)){
        std::fprintf(stderr, "par: not sorted\n");
        return EXIT_FAILURE;
    }
    if(!write_array_to_file("result.txt", work)) {
        std::fprintf(stderr, "Cannot write to file\n");
        return EXIT_FAILURE;
    }
    double S = T_seq / T_par;
    double X = S / threads;

    std::printf("Количество элементов в массиве: %d\n Максимальное количество потоков: %d\n", n, threads);
    std::printf("Время работы последовательной сортировки: %.6fсек\n", T_seq);
    std::printf("Время работы параллельной сортировки: %.6fсек\n", T_par);
    std::printf("Ускорение S = %.6f\n", S);
    std::printf("Эффективность X = %.6f\n", X);
     return EXIT_SUCCESS;
}
```

Протокол работы программы

1.
```
diana@DESKTOP-NSHR21B:~/lab2$ ./sort -f data.txt -t 2
Количество элементов в массиве: 1000000
 Максимальное количество потоков: 2
Время работы последовательной сортировки: 0.065849сек
Время работы параллельной сортировки: 0.054879сек
Ускорение S = 1.199880
Эффективность X = 0.599940
```

2.
```
diana@DESKTOP-NSHR21B:~/lab2$ ./sort -f data.txt -t 4
Количество элементов в массиве: 1000000
 Максимальное количество потоков: 4
Время работы последовательной сортировки: 0.065711сек
Время работы параллельной сортировки: 0.038365сек
Ускорение S = 1.712790
Эффективность X = 0.428197
```

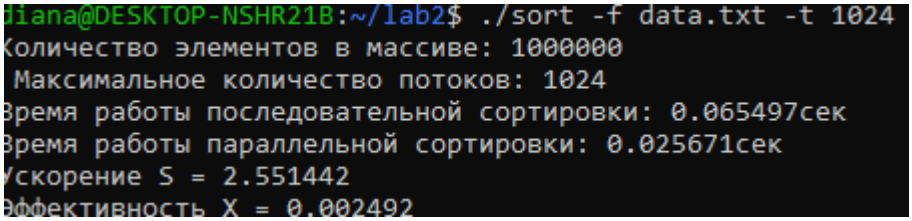
3.
```
diana@DESKTOP-NSHR21B:~/lab2$ ./sort -f data.txt -t 6
Количество элементов в массиве: 1000000
 Максимальное количество потоков: 6
Время работы последовательной сортировки: 0.066564сек
Время работы параллельной сортировки: 0.031194сек
Ускорение S = 2.133849
Эффективность X = 0.355641
```

4.
```
diana@DESKTOP-NSHR21B:~/lab2$ ./sort -f data.txt -t 10
Количество элементов в массиве: 1000000
 Максимальное количество потоков: 10
Время работы последовательной сортировки: 0.065651сек
Время работы параллельной сортировки: 0.021588сек
Ускорение S = 3.041111
Эффективность X = 0.304111
```

5.
```
diana@DESKTOP-NSHR21B:~/lab2$ ./sort -f data.txt -t 12
Количество элементов в массиве: 1000000
 Максимальное количество потоков: 12
Время работы последовательной сортировки: 0.068169сек
Время работы параллельной сортировки: 0.018935сек
Ускорение S = 3.600142
Эффективность X = 0.300012
```

6.
```
diana@DESKTOP-NSHR21B:~/lab2$ ./sort -f data.txt -t 16
Количество элементов в массиве: 1000000
 Максимальное количество потоков: 16
Время работы последовательной сортировки: 0.065531сек
Время работы параллельной сортировки: 0.018739сек
Ускорение S = 3.497102
Эффективность X = 0.218569
```

7.
```
diana@DESKTOP-NSHR21B:~/lab2$ ./sort -f data.txt -t 128
Количество элементов в массиве: 1000000
 Максимальное количество потоков: 128
Время работы последовательной сортировки: 0.065697сек
Время работы параллельной сортировки: 0.020921сек
Ускорение S = 3.140222
Эффективность X = 0.024533
```

8. 

Для оценки эффективности параллельной быстрой сортировки используются две метрики: ускорение и эффективность.

- Ускорение показывает, во сколько раз параллельный алгоритм выполняется быстрее последовательного. Находится, как частное от деления времени последовательной сортировки на время выполнения параллельной сортировки при заданном максимальном количестве потоков.

  Ускорение: $S = T_s / T_p$, где
  - T_s – время последовательной реализации,
  - T_p – время параллельной реализации,
  - $1 <= S <= p$,
  - p – количество ядер
- Эффективность показывает, насколько результативно используются потоки при параллельной обработке. Она вычисляется, как частное от деления ускорения на заданное максимальное количество потоков. (S – ускорение, p – кол-во ядер).

  Эффективность: $X = S / p$, где $X < 1$

Размер входного массива – 1 000 000 целых чисел.

Число потоков	Время последовательной сортировки, с	Время параллельной сортировки, с	Ускорение	Эффективность
2	0.065849	0.054879	1.199880	0.599940
4	0.065711	0.038365	1.712790	0.428197
6	0.066564	0.031194	2.133849	0.355641
10	0.065651	0.021588	3.041111	0.304111
12	0.068169	0.018935	3.600142	0.300012
16	0.065531	0.018739	3.497102	0.218569
128	0.065697	0.020921	3.140222	0.024533
1024	0.065497	0.025671	2.551442	0.002492

Выводы по результатам экспериментов

Наилучшее ускорение среди проведённых экспериментов было получено при ограничении в 12 потоков: время параллельной сортировки составило 0.018935с, а ускорение — 3.600142. При этом эффективность составила примерно 30%.

При ограничении в 1024 потока время сортировки увеличилось до 0.025671с, а ускорение снизилось до 2.551442. Это может объясняться ростом накладных расходов на управление потоками и конкуренцией за вычислительные ресурсы.

Можно сделать вывод, что максимальное ускорение не обязательно достигается при максимальном количестве потоков. Результаты зависят от особенностей алгоритма, объёма данных, накладных расходов и доступных вычислительных ресурсов.

Вывод

В ходе лабораторной работы была реализована последовательная и параллельная быстрая сортировка массива целых чисел с использованием библиотеки POSIX Threads. Эксперименты показали, что наибольшее ускорение (3.6 раз) было достигнуто при ограничении в 12 потоков, что соответствует количеству логических ядер. Дальнейшее увеличение до 1024 потоков привело к снижению производительности. При увеличении количества потоков эффективность их использования снижалась из-за накладных расходов на создание потоков и синхронизацию. В результате было установлено, что оптимальное количество потоков зависит от особенностей алгоритма и доступных вычислительных ресурсов.